



1

Learning Objectives

After studying this chapter, you should be able to:

- Understand Memory analysis methodology
- Conduct Process and memory dump analysis
- Discover deleted files and analyze the digital artifacts.



2

Methodology



- It is advisable for practitioners to follow a methodology to ensure that all potential evidence is uncovered and can be utilized in an incident investigation.
- One of the goals of memory analysis is to identify potentially malicious processes or executables that can be extracted and examined
- There are a variety of methodologies that can be used which depend on the type of incident.
- E.g. a methodology geared towards identifying indicators of compromise during malware infection may yield a great deal of information but may not be the best approach if the analyst has evidence from other network sources of a suspect IP address
- Some examples of methodologies are in the following slides

3

Network Connections Methodology



- In many incidents, the first indication that a system has been compromised is attempted or completed connections to external hosts detected via firewalls or web proxies
- From this starting position, it may be possible to identify potential malware on a system



4

Network Connections Methodology (Cont)



- The usual methodology to trace the operations of malware include:
- Identifying suspicious network connections – a review of network connections on hosts that have been associated with external connections will often provide details of the process that is attempting to communicate
- Identifying process name – Examining the process from the network connections allows analysts to perform similar actions found within the SANS methodology
- Identifying parent PID – insight into the parent process is useful for determining whether the process is legitimate and has a legitimate need to communicate via a network connection
- Identifying related entities – examining the associated DLLs and other artifacts brings us to the stage where they can be acquired and analyzed

5

SANS Methodology



The SANS Institution (Escal Institute of Advanced Technologies) makes use of a six-part methodology for the analysis of memory images – mainly for analysis of malware attacks



1. Identify rogue processes

- Malware often hides its behavior behind processes that on the surface may seem legitimate.
- Uncovering these involves identifying what processes are running, finding the location in the operating system they are running from, and verifying that only legitimate processes are in use.
- Sometimes processes are hidden in plain sight, and adversaries change a single letter in a process name. Other times, they will attempt to execute a process from an illegitimate source.

6

SANS Methodology



2. Analyse process DLL and handles
 - Once identified, the next step is to examine the DLL files associated with the process as well as other factors such as account information
 - DLL files are often leveraged by malware coders to hide their activity.
 - Techniques for using DLL files to compromise a system include techniques where malware coders insert their own malicious DLL files as part of the malware. Other techniques include DLL injection, where a path is written in the process to one of the malicious DLLs
3. Review network artifacts
 - Compromised systems often require communications over the Internet
 - Active and listening network connections are often contained within the memory of these systems
 - Identifying external host IP addresses may give some insight into what type of compromise has taken place and what possible activity might be planned

7

SANS Methodology



4. Look for evidence of code injection
 - Techniques such as process hollowing, and unmapped sections of the memory are often used by advanced malware coders.
 - Memory analysis tools help analysts to find evidence of these techniques
5. Check for rootkits
 - Many threat actors have goals to remain persistence in a target system
 - If they are able to achieve the initial compromise of the system, it is critical that they maintain that.
 - As a result, adversaries might use a Rootkit or malware that imbeds itself deep within the operating system and should be identified
6. Dump suspicious process and drivers
 - After locating any suspicious processes or executables, analysts need to be able to acquire them for later analysis with additional tools.

8

Other Notable Methodologies



For cybersecurity (and in part forensics) other popular methodologies include

1. OWASP (Open Web Application Security Project)
2. NIST 800-86 - Guide to Integrating Forensic Techniques into Incident Response

9

MEMORY ANALYSIS



10

Memory Analysis Overview



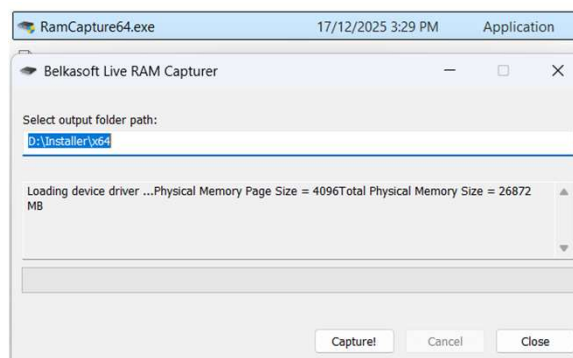
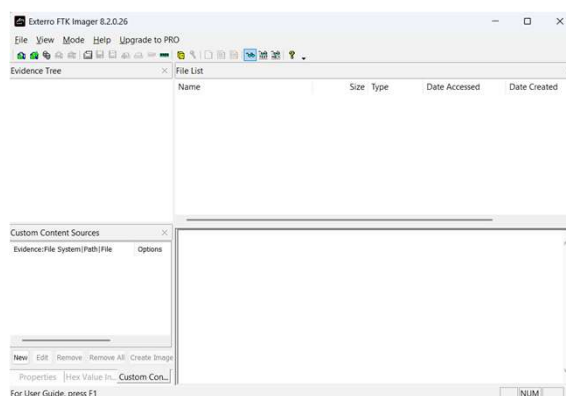
- Two interchangeable terms meaning the same thing → memory == RAM
 - The portion of a computer system(s) where the operating system places data utilized by applications/system hardware while that application or hardware is in use
 - Is volatile in nature and is lost once power is no longer supplied
- One change in operating systems that has had a direct impact on memory analysis is the advent of the 64-bit OS (majority of modern systems)
 - 64-bit addressing = 17,179,869,184 GB of memory (16 exabytes) whilst 32-bit = 4,294,967,296 = 4GB
- As a result, there is a good deal of data contained within RAM at the time a system is running that is valuable in incident investigation including running processes, loaded Dynamic Link Libraries (DLL), loaded device drivers, open registry keys, network connections, command history, etc.

11

Memory Capture (Recap)



- Covered in host-based evidence lecture (#7)
- Using tools like FTK Imager, WinPMem/LinPMem, RamCapturer, etc



12

Memory Analysis Tools



- There are many tools available to (forensic) practitioners for the review of memory images.
- Some tools provide a GUI for ease of use, while others operate via the command line, making them useful for scripting – good to determine if it is a single-time operation or bulk extraction
- Options for free tools to perform memory analysis include
 - FireEye (Mandiant) Redline
 - Volatility
 - Strings

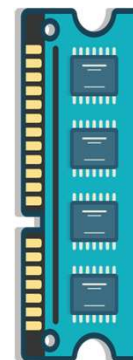
13

Memory Gems



Below is some valuable data you can find in RAM for investigations:

- | | |
|------------------------------------|---|
| 1. Network connections | 7. Command history and console sessions |
| 2. File handles and open files | 8. Kernel data structures |
| 3. Open registry keys | 9. User and credential information |
| 4. Running processes on the system | 10. Malware artifacts |
| 5. Loaded modules | 11. System configuration |
| 6. Loaded device drivers | 12. Process memory regions |



14

Approach to Memory Forensics



Identifying and Checking Processes

Start by listing all processes that are currently running. Harmful programs can pretend to be normal ones, often using names that are very similar to trick people. To handle this:

1. List every active process.
2. Find out where each one comes from in the operating system.
3. Compare them to lists of known safe processes.
4. Note any differences or odd names that stand out.

Examining Process Details

After spotting processes that might be problematic, look closely at the related dynamic link libraries (DLLs) and resources they use. Bad software can hide by misusing DLLs. Key steps include:

1. Review the DLLs connected to the questionable process.
2. Look for any that are not approved or seem harmful.
3. Check for evidence of DLLs being inserted or taken over improperly.

15

Approach to Memory Forensics (Cont)



Reviewing Network Connections

A lot of malware needs to connect to the internet, such as to contact control servers or send out stolen information. To find these activities:

1. Check the open and closed network links stored in memory.
2. Record any outside IP addresses and related web domains.
3. Figure out what the connection is for and why it's happening.
4. Confirm if the process is genuine.
5. See if it usually needs network access.
6. Track it back to the process that started it.
7. Judge if its actions make sense.

Finding Code Injection

Skilled attackers may use methods like replacing a process's code or working in hidden memory areas. To detect this:

1. Apply tools for memory analysis to spot unusual patterns or signs of these tactics.
2. Point out processes that use strange memory locations or act in unexpected ways.

16

Approach to Memory Forensics (Cont)



Detecting Rootkits

Attackers often aim for long-term access and hiding. Rootkits bury themselves deep in the system, giving high-level control while staying out of sight. To address them:

1. Search for indicators of rootkit presence or major changes to the OS.
2. Spot any processes or drivers with extra privileges or hidden traits.

Isolating Suspicious Items

Once suspicious processes, drivers, or files are identified, pull them out for further study. This means:

1. Extract the questionable parts from memory.
2. Save them safely for detailed review with forensic software.

17

The Volatility Framework



- A widely recommended option for memory forensics is Volatility.
 - This is a prominent open-source framework used in the field. Its main component is a Python script called Volatility, which relies on various plugins to carefully analyze memory dumps.
 - Since it is built on Python, it can run on any system that supports Python.
- Volatility's modules, also known as plugins, are additional features that expand the framework's capabilities.
 - They help pull out particular details or carry out targeted examinations on memory files.

18



Frequently Used Volatility Modules

- **pslist**: Shows the active processes.
- **cmdline**: Reveals the command-line parameters for processes.
- **netscan**: Checks for network links and available ports.
- **malfind**: Looks for possible harmful code added to processes.
- **handles**: Examines open resources.
- **svcsan**: Displays services in Windows.
- **dlllist**: Lists the dynamic-link libraries loaded in a process.
- **hivelist**: Identifies registry hives stored in memory.

19

SYSTEM STORAGE ANALYSIS



20

Overview of System Storage Analysis



- Dead recovery (as covered in earlier lectures) create dumps of system/tertiary storage which can be further duplicated and analysed without risking the original evidence source
- Actual system storage analysis is a complex process and the depth and breadth of it cannot be explored in a single lesson
- We'll target several areas of focus with the understanding that forensic practitioners will gain a better sense of some of the operations performed and tools that can be employed, as well as an understanding of some of the critical data that can be leveraged.

21

Forensic Platforms



- Like other digital forensic tools, the ones for storage analysis have been expanding in terms of functionality and ease of use along with price
- The limiting factor in utilizing many of these platforms is now mostly due to the cost, however a lower-cost alternative will be just as effective for an incident response team that only occasionally needs the functionality provided
- For legality purposes, several organizations do test platforms for efficacy, such as the NIST's Computer Forensic Tools Testing Program
- As a reminder, there (currently?) is no single court accepted standard, but tools should conform to the conventional rules of evidence
 - The use of a platform that has not been tested or does not conform to the rules of evidence may lead to the evidence being excluded from legal proceedings.

22

Forensic Platforms Features



Regardless of type, a storage analysis platform will have features like

- **File structure view** – should have the ability to view the file structure and allow analysts to quickly review and locate files on a suspect system
- **Hex viewer** – allows practitioners to obtain a granular look at files which may be beneficial in cases involving malware or custom exploits
- **Web artifacts** – platforms with ability to examine web traces are useful in attacks involving malicious websites or social engineering attacks
- **Email carving** – cases involving illegal activity or policy violations often involve emails and having a platform that can pull this data out assists the analyst in viewing communication between the suspect system and others
- **Image viewer** – it is necessary to view the images that are saved on systems especially in cases involving photographic evidence
- **Metadata** – key pieces of data about files that are useful for depicting conditions on a suspect device (e.g. time, date, location, etc)

23

Example of Platform

A repeat from previous lectures

- OpenText EnCase
- Exterro Forensic Toolkit
- X-Ways Forensics
- SANS Forensics
- Autopsy/SleuthKit



24

Storage Analysis



- Storage analysis can be performed to extract items/evidence/information such as (amongst many others):
 - Deleted files identification and recovery / carving
 - Keyword searching
 - Attached devices history
 - Timeline analysis
 - Registry/Log analysis



25

File Carving



- Process used in digital computer forensics to extract data from a disk drive or other storage device
- File carving is most often used to recover files from the unallocated space in a drive – occurs when files are deleted but the actual file content still remains and is accessible by examining file headers/footers to extract content
- File carving is a great method for recovering files and fragments of files when directory entries are corrupt or missing
- While often mistaken with file recovery, file carving does not rely on the assistance of the file system that originally created the file (i.e. does not care about underlying storage method)
- Note that, although less common, carving can be done on memory dumps

26

File Carving Methods



- Three common methods in use to perform carving
 - Header-footer / Header-filesize carving (Signature-based)
 - Extracts files based on header/footer boundaries or header and file size only
 - Structure-based carving
 - Extracts files with combination of header/footer and metadata that indicates file system structure (e.g. filename, folder name, address location, block number etc)
 - Content-based carving
 - Extraction using content of files which may be fragmented in storage. Recovers files by analysing the contents of a target area which then performs identification, validation, and reassembling the fragmented portions of the files → good for image recovery

27

Tools for File Carving

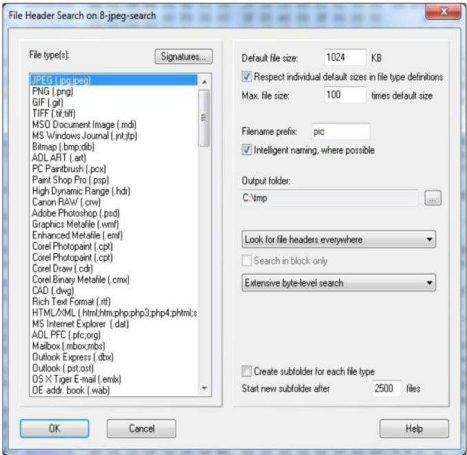


- Many tools available to perform file carving including
 - Foremost / Scalpel
 - FTK + plugins
 - bulk_extractor
 - EnCase
 - X-Way WinHex
 - EVTXtract → specialized for Windows event logs



28

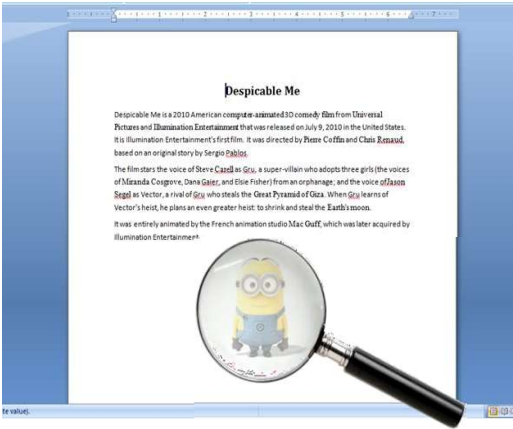
Sample Tools



- Some tools (WinHex shown here) have filters and limits to avoid the aforementioned problems with file carving
- Known header signatures are built-in for selection and file sizes can be set (to avoid problems of filetypes with no footer)

29

Sample Tools

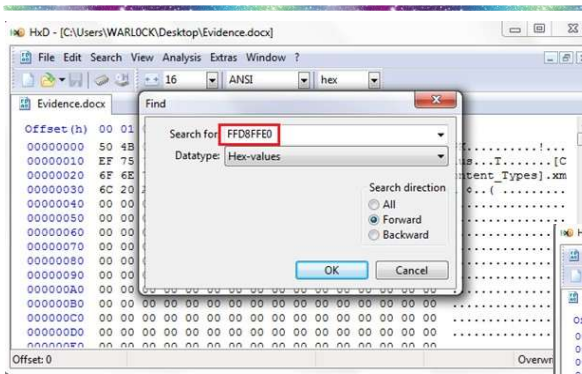


- Sometimes even a hex editor can be used to carve files from storage (i.e. if you know the header and footer bytes) or even other containers/documents
- This process is obviously much slower, and manual compared to using tools such as those listed above
- The following slides shows a good example from Infosec with a Word doc file with embedded image (JPG filetype)

30

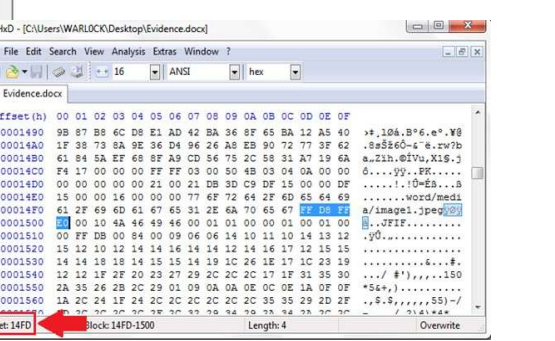


Sample Tools



Once located, the offset address is identified and recorded for later use (in this example block 14FD in the document)

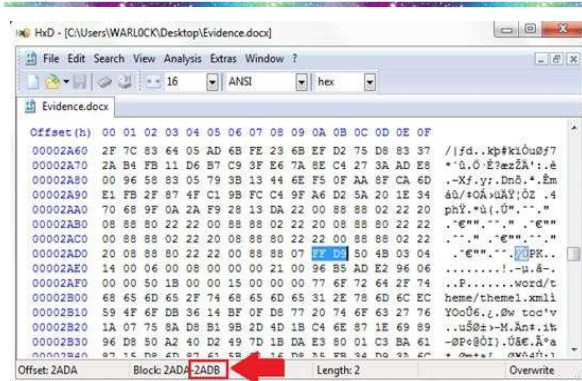
JPEG files have a common hex header of FFD8FFE0 So a search for that string is done using a hexeditor (HxD in this case)



31

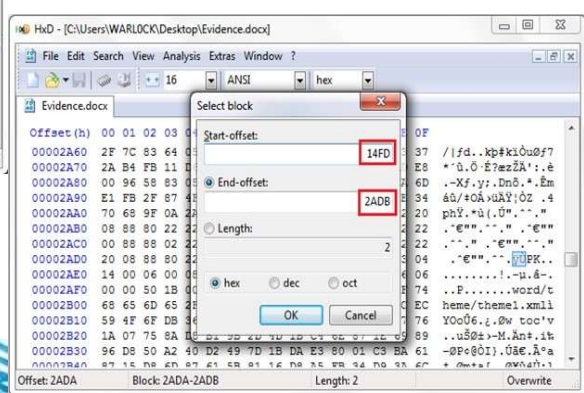
Sample Tools





The select function in HxD is used to highlight the start and end of file using the offsets obtained earlier. Then the bytes can be copied and saved

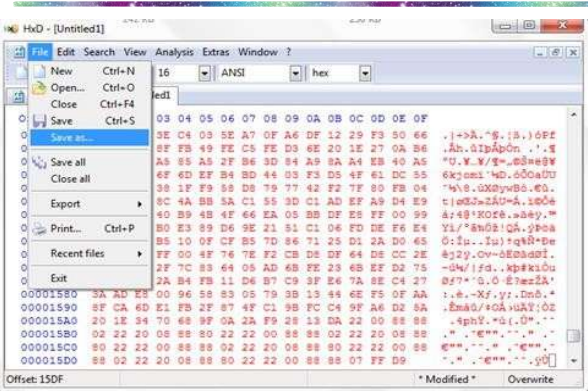
Process is repeated with JPEG footer (FFD9) and the offset address is recorded as the EOF (2ADB in this example)



32

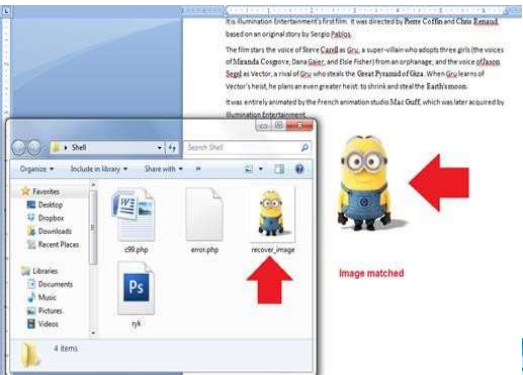


Sample Tools



The bytes in question are extracted and saved

If contiguous and data is not corrupted, the entire embedded file can be extracted from its container



33

File Carving Issues



- File carving could possibly be a time-consuming process, especially if the analyst has no idea what to look for
- Many tools rely on data that remains intact and contiguous and do not work well on fragmented files
 - Heavily active storage devices with many read/write actions complicate recovery
 - Improving with content-based but still lacking
- Many files actually do not have a footer, repeated headers, changing signatures, etc. making carving difficult
- Actual carving is known to be a problem with a time complexity of NP- complete → impossible to solve and verify in reasonable amount of time

34

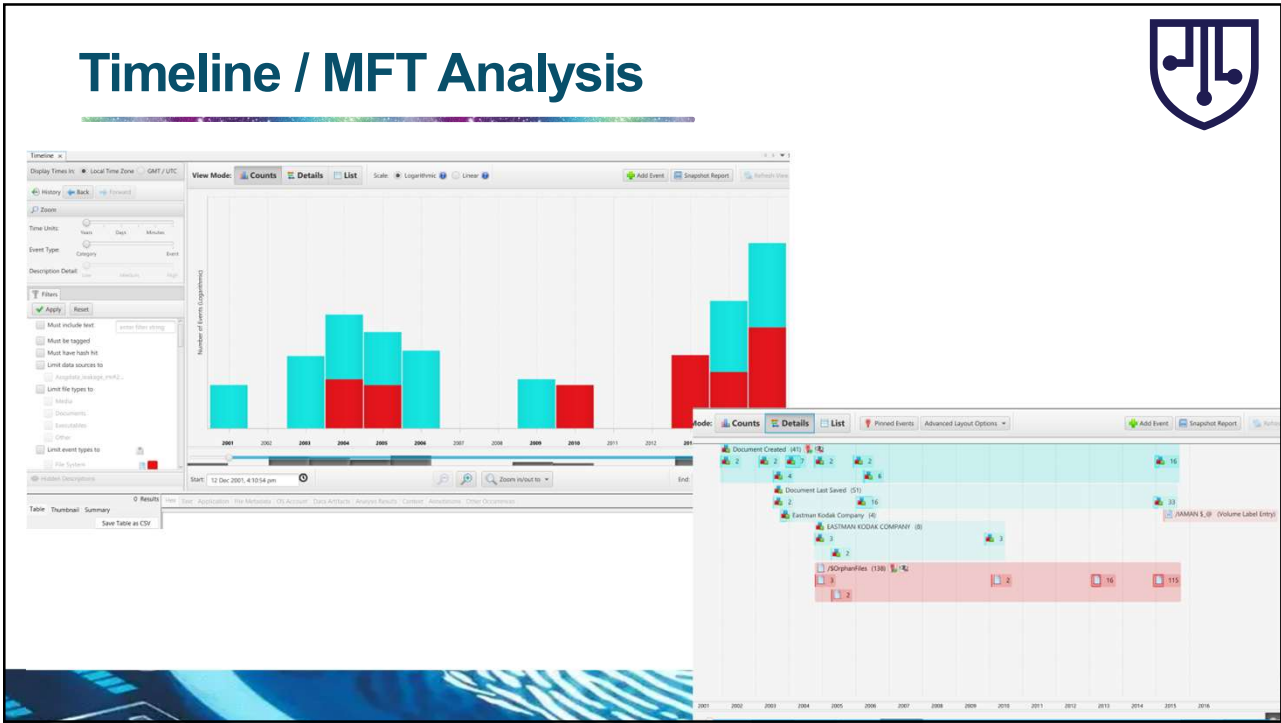
Timeline / MFT Analysis



- When investigating an incident, it is sometimes critical to have an idea of when applications or files were executed
- Timestamps can sometimes be found in other aspects of the investigation, such as when examining memory images.
- Also, identifying specific DLL files or executable files in a memory image can be compared to the date and time they were accessed in order to correlate other activity that's been observed on the system
- Regardless, forensic practitioners have to ensure system dates are correct before analyzing for timelines
- Some platforms (e.g. Autopsy) have a timeline function that maps access times of files against a timeline

35

Timeline / MFT Analysis



36

Registry Analysis



- Since majority of devices are using Windows as its operating system, a bit portion of analysis on storage devices are the registry entries created key-value pairs that might hold vital information.
- These registry hives are config folder on the Windows file structure consisting of HKEY_CURRENT_USER, HKEY_USERS, HKEY_CLASSES_ROOT, HKEY_LOCAL_MACHINE, and HKEY_CURRENT_CONFIG
- HKLM is usually the target as it contains the SAM (passwords hashed), security logs, software logs, and system logs. An alternative is the HKCU
- Changes that have been made to the user's data are recorded in that user's NTUSER.dat file → contains profile settings, network connections, connected systems

37

Registry Analysis



VALUES TYPES STATES

HIVES
KEYS
SUBKEYS

Name	Type	Data
(Default)	REG_SZ	(value not set)
BootDriverFlags	REG_DWORD	0x0000001c (28)
CurrentUser	REG_SZ	USERNAME
EarlyStartServices	REG_MULTI_SZ	RpcSs Power BrokerInfrastructure System
FirmwareBootD...	REG_SZ	multi(0)disk(0)rdisk(0)partition(2)
LastBootShutdo...	REG_DWORD	0x00000001 (1)
LastBootSuccee...	REG_DWORD	0x00000001 (1)
PreshutdownOr...	REG_MULTI_SZ	UsoSvc DeviceInstall gpssvc trustedinstall
SvcHostSplitThr...	REG_DWORD	0x00380000 (3670016)
SystemBootDevi...	REG_SZ	multi(0)disk(0)rdisk(0)partition(4)
SystemStartOpti...	REG_SZ	NOEXECUTE=OPTIN NOVGA
WaitToKillServ...	REG_SZ	5000

SUBKEY CONTENT

38

19

Extracting/Exporting Registry Hives



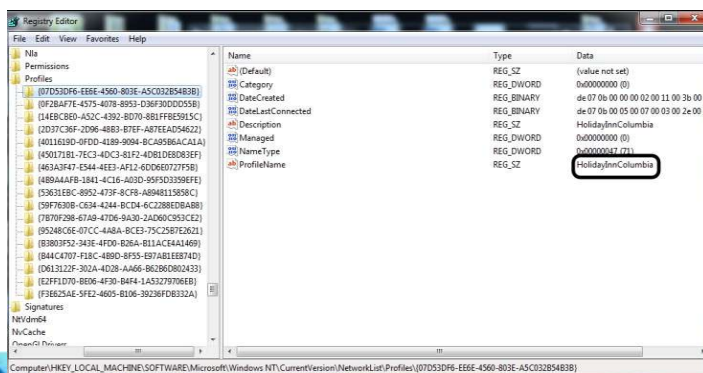
- Registry editor export
 - As reg files (individual keyvalue pairs)
 - As hive files (no extension) which are binary files
 - As text files
- Can be done using tools like FTK Imager that can read the registry in storage images OR through [reg save] operations that can capture live system registry values
- There is so much information possibly stored in registry that we cannot cover in a single lecture – you are left to explore this on your own.

39

Other Examples of Registry Information



- Most recently used/installed/deleted software and last files accessed
- All devices mounted on the system
- Search operations and search terms done on the system
- Known connection points (i.e. network connections), last IP, DNS records etc



40

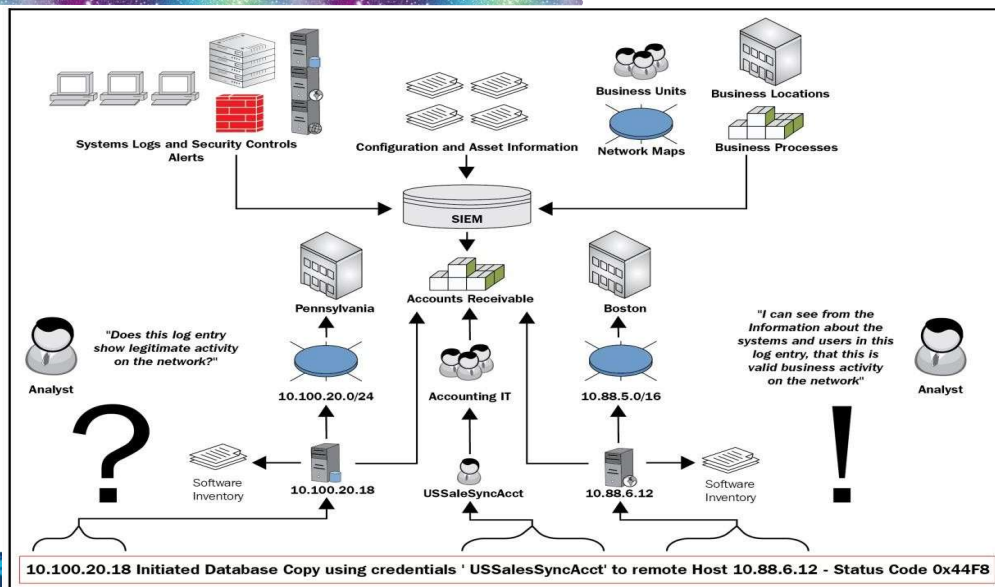
Log File Analysis



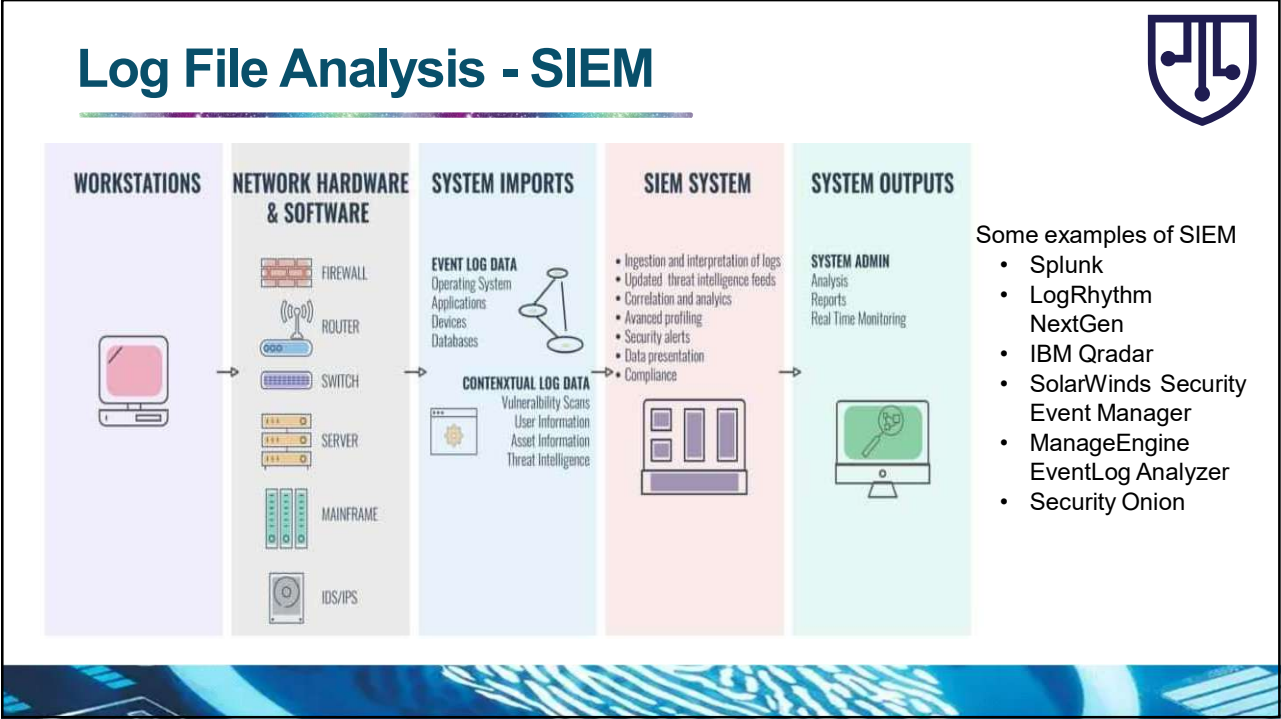
- This has also been partially covered in tutorial sessions
- Majority of it focuses on Windows event logs
 - For the casual user the logs might not be available but for many corporations, proper IT practices would have some form of logging and auditing on company hardware in place (especially for centralized accessed hardware and publicly accessible content)
 - One tool that has been embraced by a number of enterprises is a SIEM (Security Information and Event Management) system
 - These appliances have the ability to aggregate log and event data from network sources and combine them into a single location for easier monitoring and analysis should an event occur
- Linux systems also have logs (usually found under /var/log) that can be accessed via dmesg

41

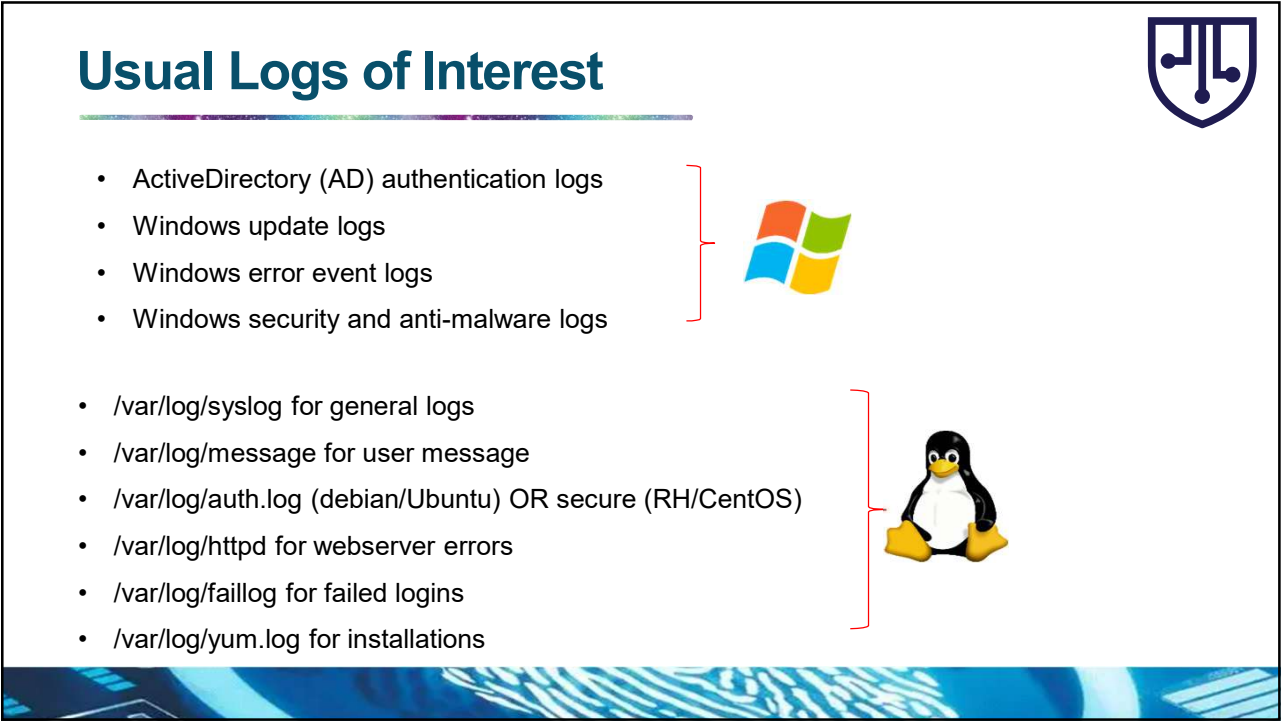
Log File Analysis - SIEM



42



43



44

Key Notes



- Analysis of memory and storage is made with the assumption that any actions by an adversary will leave a trace behind
- These traces rely on the forensic practitioners' ability to see these traces is based upon the tools and techniques that are used
- Additionally, experience with the content and tools also are needed especially with a ever-changing and constantly updating environment
- SIM/SEM/SIEM tools are not foolproof and cannot replace human monitoring for suspicious behavior on business networks

45

Summary



What was covered today:

- Overview in the different methodologies and tools used in both analysis of memory captures and storage captures (images)
- Although both may use different (or possibly similar) tools and methods, it is important to remember that a corroboration between evidence collected from different sources is needed to support case findings
- The overall content covered here only scratches the surface of all operations possible in both domains there are numerous resources available (both paid and free) to explore



46

